Fundamentals of Python Programming

Table of Contents

1. Introduction and Learning Goals 5
2. Built-in Data Types 5
   2.1 Primitive-like Types: int, float, bool, str 5
      Integers and floating-point numbers 5
      Boolean values 6
      Strings 6
   2.2 Collections: list, tuple, dict, set 6
      List 7
      Tuple 7
      Dictionary 7
      Set 7
   2.3 Objects, References, Mutability, and Type Checking 8
3. Literals in Python 8
   3.1 Numeric, String, Boolean, None, and Collection Literals 8
   3.2 Escape Sequences and Formatting 9
4. Variables, Scope, and Attributes 9
   4.1 Variables and Local Variables 9
      Scope: LEGB 10
   4.2 Instance Attributes and Class Attributes 10
      Mutable class-attribute pitfall 11
   4.3 Student Class Example 11
5. Naming Conventions, Access Control, and Methods 12
   5.1 Public, Non-Public, and Private-Style Names 12
   5.2 Name Mangling and Encapsulation 12
      Validation as part of encapsulation 13
   5.3 The @staticmethod Decorator 13
   The @classmethod Decorator 14
6. Abstract Classes and Other OOP Features 14
   6.1 ABC and @abstractmethod 14
   6.2 Encapsulation and @property 15
      Property pitfalls 15
   6.3 @final and typing.final 16
   6.4 Shape, Circle, and Rectangle Example 16
7. Operators — Part 1 17
   7.1 Arithmetic Operators 17
   7.2 Comparison Operators 18
   7.3 Logical Operators 18
      Truthiness 19

7.4 Bitwise Operators 19
8. Operators — Part 2 19
8.1 Assignment Operators 19
8.2 Identity, Membership, and Conditional Expressions 20
Identity: is and is not 20
Membership: in and not in 20
Conditional expression 21
8.3 Precedence and Associativity 21
9. Decision-Making 22
9.1 if, if-else, and if-elif-else 22
Boundary conditions 22
9.2 Compound Conditions and Validation 23
9.3 Nested if Statements 23
9.4 Multiple-Branch Selection with match-case 23
10. Loops and Loop Control 24
10.1 The while Loop 24
Input validation with while 24
10.2 The for Loop 25
Iterating through dictionaries 25
Calculating an average 25
10.3 No Native do-while Loop 26
10.4 break, continue, pass, and Loop else 26
Loop else 26
Common loop mistakes 27
11. Python and Java: Important Differences 27
Conditional expression comparison 28
Loop comparison 29
12. Common Pitfalls and Debugging Checklist 29
12.1 Data Types and Literals 29
12.2 Variables and OOP 29
12.3 Operators and Conditions 29
12.4 Loops 30
12.5 Practical Debugging Checklist 30
13. Practice Exercises 30
Exercise 1 — Types and literals 30
Exercise 2 — Variables and attributes 30
Exercise 3 — Access conventions and methods 30
Exercise 4 — Abstract class and property 31
Exercise 5 — Operators 31
Exercise 6 — Decisions 31

Exercise 7 — Loops....31
Exercise 8 — Reflection....31
15. Conclusion and Quick Reference....31
Suggested Official References .... **Error! Bookmark not defined.**

1. Introduction and Learning Goals

Python is a high-level language with readable syntax. It supports procedural programming and object-oriented programming (OOP). In OOP, classes define the structure and behavior of objects. Python is dynamically typed: names refer to objects, and a name can later refer to an object of another type.

- Choose suitable types for numbers, text, Boolean values, and collections.
- Distinguish literals, variables, references, local variables, instance attributes, and class attributes.
- Explain Python naming conventions without treating them as strict security controls.
- Use static methods, class methods, abstract methods, properties, and final-type hints correctly.
- Use arithmetic, comparison, logical, bitwise, assignment, identity, and membership operators.
- Write conditions with if, elif, else, nested if, and match-case.
- Use for and while loops, plus break, continue, and pass.
- Identify common mistakes and explain important Python–Java differences.

2. Built-in Data Types

2.1 Primitive-like Types: int, float, bool, str

Python calls these built-in types. The word “primitive” is common in Java, but Python's values are objects, including integers and Booleans.

int	Whole number	age = 19	Can be positive, negative, or zero.
float	Floating-point number	score = 87.5	Stores approximate binary values, so some decimal calculations are not exact.
bool	Truth value	passed = True	Only True and False; bool is a subclass of int.
str	Text	name = 'Dara'	Unicode text; immutable sequence of characters.

Integers and floating-point numbers

introduct_to_oop.py

```python
age = 19
year = 2026
score = 87.5
temperature = -2.25
print(age + 1)        # 20
print(7 / 2)          # 3.5
print(7 // 2)         # 3
print(type(score))    # <class 'float'>
```

The ‘/’ operator performs true division in ordinary numeric operations. The “//” operator performs floor division, rounding down toward negative infinity. Therefore, -7 // 2 is -4, not -3.

introduct_to_oop.py

```
print(0.1 + 0.2)            # commonly 0.30000000000000004
print(round(0.1 + 0.2, 2))  # 0.3
```

Floating-point values are approximations. For applications that require exact decimal arithmetic, such as financial calculations, consider decimal. For many beginner exercises, float is sufficient.

Boolean values

introduct_to_oop.py > ...

```
is_enrolled = True
has_paid_fee = False

print(is_enrolled and has_paid_fee)  # False
print(int(True))                     # 1
print(int(False))                    # 0
```

Boolean literals are case-sensitive: write **True** and **False**, *not true and false*. Because bool is a subclass of **int**, True behaves like 1 and False like 0 in some numeric contexts. This is useful to know, but avoid relying on it when a real Boolean is not intended.

Strings

introduct_to_oop.py > ...

```
student_name = "Menghean"
message = "Welcome, " + student_name
print(message)   #Welcome, Menghean

print("Python"[0])        # P
print("Python"[-1])       # n
print("Python".lower())   # python
print(len("Python"))      # 6
```

String indexing **begins at zero**. Negative indexing *counts from the end*. Strings are immutable, so methods such as lower() return a new string rather than changing the original object.

2.2 Collections: list, tuple, dict, set

Type	Syntax	Ordered?	Mutable?	Duplicates?	Typical purpose
list	[1, 2, 3]	Yes	Yes	Yes	Sequence that can be changed.
tuple	(1, 2, 3)	Yes	Items cannot be reassigned	Yes	Fixed sequence or record.
dict	{'name': 'Dara'}	Insertion order preserved	Yes	Keys are unique	Map keys to values.

set	{1, 2, 3}	No positional order guarantee	Yes	No duplicates	Unique values and set operations.

List

```python
names = ["Dara", "Sokha", "Menghean"]
names.append("Malis") # add "Malis" into names
print(names)   # ['Dara', 'Sokha', 'Menghean', 'Malis']
names[0] = "Dara S."  # Change the value in the index [0]
print(names)  # ['Dara S.', 'Sokha', 'Menghean', 'Malis']
print(names[1])  # Sokha
print(names[1:3]) # slice: items at indexes 1 and 2 -> #['Sokha', 'Menghean']
```

Lists support indexing, slicing, appending, removing, and replacing elements. They may contain different types, but using one consistent type is often easier to maintain.

Tuple

```python
location = (11.56, 104.93)
record = ("S001", "Dara", 19, 88.5)
one_item_tuple = ("Python",)
print(record[1])  # Dara
# Tuple is immutable, so you can not reassign or change the value in tuple
```

A tuple is immutable at the tuple level. You cannot assign a different object to record[1]. A tuple can, however, contain a mutable object such as a list; that inner list can still be modified. The comma is what makes a one-item tuple.

Dictionary

```python
student = {
    "id": "S001",
    "name": "Dara",
    "age": 19,
    "score": 88.5
}
print(student)       #{'id': 'S001', 'name': 'Dara', 'age': 19, 'score': 88.5}
print(student["name"])   # Dara
student["score"] = 91
student["email"] = "dara@example.com" # add new key_value pair into dictionary student
print(student) #{'id': 'S001', 'name': 'Dara', 'age': 19, 'score': 91, 'email': 'dara@example.com'}
print(student.get("phone"))  # None if key is missing, if we do not use the function 'get()' it will error
```

A dictionary stores key-value pairs. Accessing a missing key with student['phone'] raises KeyError; get() can return None or a supplied default. Dictionary keys must be hashable. If the same key is repeated in a dictionary literal, the later value replaces the earlier one.

Set

```python
subjects = {"Python", "Math", "English", "Math"}
print(subjects)  # each item appears once; display order may vary
# {'Python', 'Math', 'English'} , 'Math' has been remove from the set
required = {"Python", "Math"}
completed = {"Math", "Python", "Statistics"}
print(required.issubset(completed))  # True
print(required & completed)          # intersection
print(required.intersection(completed))   #{'Math', 'Python'}
print(required   completed)          # difference
print(required.difference(completed)) #set()
empty_set = set()                    # correct empty set
```

An empty pair of braces {} creates an empty dictionary, not an empty set. Set elements must be hashable, so a list cannot be placed directly inside a set.

2.3 Objects, References, Mutability, and Type Checking

A variable is a name bound to an object. Assignment does not automatically copy a list or dictionary.

introduct_to_oop.py > ...

```python
a = [10, 20]
b = a
b.append(30)
print(a)  # [10, 20, 30]: a and b refer to the same list, so when you append 30 to b it's also change a
c = a.copy() # c does not refere to the same list, it is new list that have element the same as 'a'
c.append(40)
print(a)  # [10, 20, 30]
print(c)  # [10, 20, 30, 40]

```

A shallow copy creates a new outer container, but nested mutable objects may still be shared. Use copy.deepcopy() only when a recursive independent copy is genuinely needed.

introduct_to_oop.py > ...

```python
value = 12
print(type(value))              # <class 'int'>
print(isinstance(value, int))   # True
print(isinstance(True, int))    # True, because bool subclasses int

```

Important note: Use == to compare values. Use is to test whether two names refer to the same object. The standard None check is `value is None`.

3. Literals in Python

A literal is a value written directly in source code. In `age = 19`, 19 is a literal and age is a variable name.

3.1 Numeric, String, Boolean, None, and Collection Literals

Category	Examples	Notes
Integer	0, 42, -8, 0b1010, 0o12, 0x2A	Binary, octal, and hexadecimal prefixes are supported. Underscores can improve readability: 1_000_000.
Float	3.14, -0.5, 2e3, 1.2e-4	Scientific notation; 2e3 means 2000.0.
Complex	2+3j	j represents the imaginary unit.
String	'Hello', "Hello", '"multiple lines"'	Single and double quotes work; triple quotes can span lines.
Boolean	True, False	Case-sensitive.
None	None	Represents absence of a value; distinct from 0, False, and empty text.
List	[1, 2, 3]	Mutable sequence.
Tuple	(1, 2, 3), ('S001',)	A one-item tuple requires a comma.

Dictionary	{'name': 'Dara'}	Key-value pairs.
Set	{1, 2, 3}	Unique values; use set() for an empty set.

3.2 Escape Sequences and Formatting



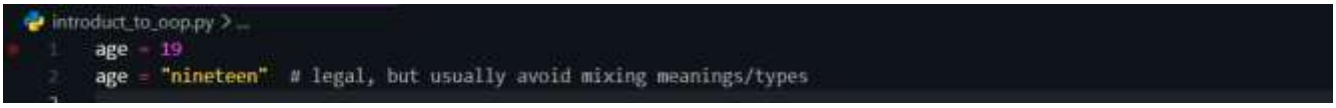
Common escape sequences include “\n” for a new line, “\t” for a tab, “\\” for a backslash, and escaped quotes. Raw strings are useful for Windows paths and regular expressions, but a raw string cannot end with one lone backslash.

- Use ('only item',) for a one-item tuple; ('only item') is just a string in parentheses.
- Use None for an absent value and compare with is None.
- Python 3 does not allow decimal integer literals with leading zeros such as 08.
- Concatenating a string and integer with + raises TypeError unless you convert the number or use an f-string.
- Duplicate dictionary keys in a literal keep the last assigned value.

4. Variables, Scope, and Attributes

4.1 Variables and Local Variables

Python creates a variable name when it is assigned. A declaration such as `int age` is not normally required. A name may be rebound to a value of another type, although doing so can make code confusing.



A local variable is a name assigned inside a function. It is normally usable only inside that function.

introduct_to_oop.py > ...

```python
age = 20 # Global variable because we create it outside the function
Tabnine | Edit | Test | Explain | Document
def calculate_average(scores):
    total = sum(scores)  # local
    count = len(scores)  # local
    if count == 0:
        return None
    average = total / count # total and count can call to use here because it is in the function
    return average

print(calculate_average([80, 90, 100]))
# print(total)  # NameError: total is local to the function
```

Scope: LEGB

Python resolves names using the LEGB rule: Local, Enclosing function, Global module, and Built-in. `global` allows a function to rebind a module-level name. `nonlocal` allows a nested function to rebind a name in an enclosing function.

introduct_to_oop.py > ...

```python
school = "ITC"

Tabnine | Edit | Test | Explain | Document
def show_school():
    local_message = "Welcome"
    print(local_message, school)

Tabnine | Edit | Test | Explain | Document
def change_school():
    global school
    school = "Another School"

show_school()
change_school()
print(school)
```

Important note: Passing values as parameters and returning results is often easier to test than changing global variables.

4.2 Instance Attributes and Class Attributes

An instance attribute belongs to one object, commonly assigned with self.attribute inside __init__. A class attribute is defined in the class body and is available to instances unless an instance attribute with the same name shadows it.

introduct_to_oop.py > ...

```python
class Student:
    school_name = "Institute of Technology of Cambodia"  # class attribute

    Tabnine | Edit | Test | Explain | Document
    def __init__(self, name, age):
        self.name = name  # instance attribute
        self.age = age

s1 = Student("Dara", 19)
s2 = Student("Malis", 20)
print(s1.name)        # Dara
```

```
print(s2.name)         # Malis
print(s1.school_name)  # shared class value

Student.school_name = "ITC"
print(s2.school_name)  # ITC
s1.school_name = "Private School"
print(s1.school_name) # Private School (instance attribute shadows class)
print(s2.school_name) # ITC
```

Assigning `s1.school_name` creates or updates an attribute on s1. It does not change the class attribute used by other instances.

Mutable class-attribute pitfall

```
class BadClass:
    grades = []  # one shared list for every instance

class BetterClass:
    Tabnine | Edit | Test | Explain | Document
    def __init__(self):
        self.grades = []  # separate list for each instance

a = BetterClass()
b = BetterClass()
a.grades.append(95)
print(a.grades)  # [95]
print(b.grades)  # []
```

4.3 Student Class Example

introduct_to_oop.py > ...

```
class Student:
    school_name = "ITC"
    student_count = 0

    Tabnine | Edit | Test | Explain | Document
    def __init__(self, student_id, name, age, score):
        self.student_id = student_id
        self.name = name
        self.age = age
        self.score = score
        Student.student_count += 1

    Tabnine | Edit | Test | Explain | Document
    def passed(self):
        return self.score >= 50

    Tabnine | Edit | Test | Explain | Document
    def summary(self):
        return f"{self.student_id}: {self.name}, age {self.age}, score {self.score}"

s = Student("S001", "Dara", 19, 88)
print(s.summary())    #S001: Dara, age 19, score 88
print(s.passed())     #True
print(Student.student_count) #1
```

The __init__ method initializes an instance after it has been created. The first parameter self refers to the instance. Python supplies it automatically when calling an instance method through an object.

5. Naming Conventions, Access Control, and Methods

5.1 Public, Non-Public, and Private-Style Names

Python does not use ordinary Java-style public, protected, and private modifiers for attributes. It relies on naming conventions and, for double-leading-underscore names, name mangling.

Style	Example	Meaning
Public	self.name	Intended for normal use by other code.
Non-public convention	self._balance	Single leading underscore signals internal use; it is not enforced privacy.
Name-mangled private-style	self.__pin	Two leading underscores, not two trailing underscores, trigger name mangling in a class.

Access Modifiers	Same Class	Same Package	Sub Class	Other Packages
Public	*Y*	*Y*	*Y*	*Y*
Protected	*Y*	*Y*	*Y*	*N*
Private	*Y*	*N*	*N*	*N*

Names such as __init__ and __repr__ are special methods (often called dunder methods). Avoid inventing double-underscore names because Python may reserve such patterns for language features.

5.2 Name Mangling and Encapsulation

introduct_to_oop.py > ...

```python
class BankAccount:
    bank_name = "Community Bank"

    Tabnine | Edit | Test | Explain | Document
    def __init__(self, owner, balance, pin):
        self.owner = owner
        self._balance = balance
        self.__pin = pin

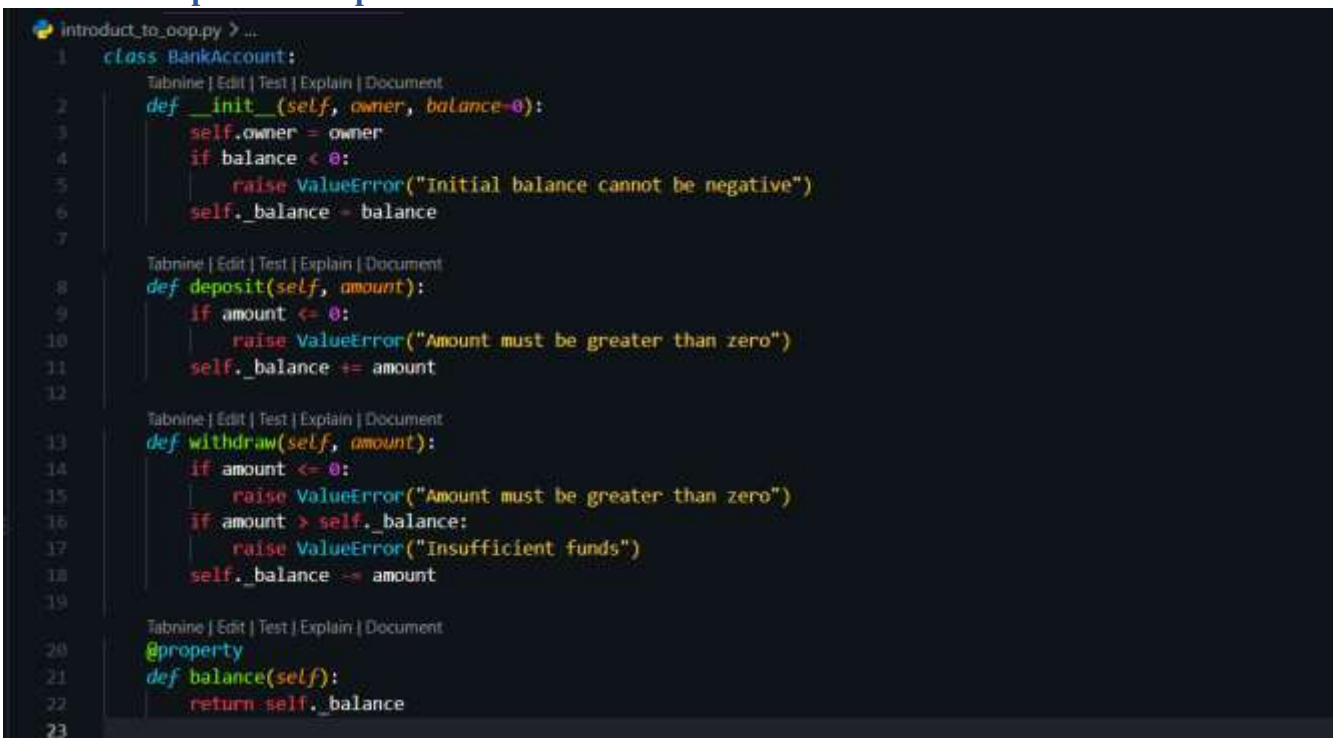
    Tabnine | Edit | Test | Explain | Document
    def deposit(self, amount):
        if amount <= 0:
            raise ValueError("Deposit must be positive")
        self._balance += amount

    Tabnine | Edit | Test | Explain | Document
    def get_balance(self):
        return self._balance

account = BankAccount("Dara", 100, "2468")
account.deposit(50)
print(account.get_balance())  # 150
print(account.owner)          # Dara
# print(account.__pin)        # AttributeError
```

The private-style attribute is usually stored under a transformed name similar to _BankAccount__pin. Name mangling helps avoid accidental name clashes in subclasses; it is not encryption or a security boundary. Do not store secrets assuming this mechanism protects them.

Validation as part of encapsulation



Encapsulation groups data and behavior and provides a controlled interface. A leading underscore recommends using the public property or methods instead of directly changing _balance. Python does not prevent all direct access.

5.3 The @staticmethod Decorator

A static method is related to a class but receives neither self nor cls automatically. It is suitable for a helper that needs no instance or class state.



The explicit Boolean rejection matters because bool is a subclass of int. Without `not isinstance(score, bool)`, True would pass an integer type check.

The @classmethod Decorator

A class method receives the class as its first parameter, conventionally called cls. It is useful for class-level behavior and alternative constructors. Calling it through a subclass passes the subclass as cls.



Feature	Instance method	Static method	Class method
Decorator	Usually none	@staticmethod	@classmethod
First automatic argument	self	None	cls
Instance state available directly?	Yes	No	No, unless passed in
Class state	Yes	Only if referenced explicitly	Yes, through cls
Common use	Object behavior	Related utility	Alternative constructor/class-level operation

Important note: Choose an instance method when behavior depends on one object; a class method when it depends on the class or constructs cls; a static method when neither self nor cls is needed.

6. Abstract Classes and Other OOP Features

6.1 ABC and @abstractmethod

An abstract class defines a common interface for subclasses. An abstract method declares an operation that a concrete subclass must implement before it can be instantiated. Python provides ABC and abstractmethod in the abc module.

```python
introduct_to_oop.py > ...
from abc import ABC, abstractmethod

class Shape(ABC):
    Tabnine | Edit | Test | Explain | Document
    @abstractmethod
    def area(self):
        pass  # placeholder body; subclass must implement this method

class Circle(Shape):
    Tabnine | Edit | Test | Explain | Document
    def __init__(self, radius):
        if radius < 0:
            raise ValueError("Radius cannot be negative")
        self.radius = radius

    Tabnine | Edit | Test | Explain | Document
    def area(self):
        return 3.141592653589793 * self.radius ** 2

circle = Circle(2)
print(circle.area())
# shape = Shape()  # TypeError: abstract class cannot be instantiated
```

The pass statement does nothing. It is acceptable as a placeholder body for an abstract method. A subclass that fails to implement every inherited abstract method remains abstract and cannot be instantiated.

6.2 Encapsulation and @property

A property gives attribute-like access while allowing a getter, setter, or deleter to run. It is useful when reading or updating a value needs validation or calculated behavior.

```python
introduct_to_oop.py > ...
class Student:
    Tabnine | Edit | Test | Explain | Document
    def __init__(self, name, score):
        self.name = name
        self.score = score  # calls the property setter

    @property
    def score(self):
        return self._score

    Tabnine | Edit | Test | Explain | Document
    @score.setter
    def score(self, value):
        if not isinstance(value, (int, float)) or isinstance(value, bool):
            raise TypeError("Score must be a number")
        if not 0 <= value <= 100:
            raise ValueError("Score must be between 0 and 100")
        self._score = value

s = Student("Dara", 85)
print(s.score)  # getter
s.score = 92    # setter
# s.score = 150  # ValueError
```

The public property is score; the backing field is _score. A property without a setter is read-only through that property, although the underlying object may still expose other ways to change data.

Property pitfalls

- Do not assign self.score inside its own setter; this calls the setter again and causes infinite recursion. Assign self._score instead.
- Avoid using the public property name as its own backing field.

- Validate values at the point where they enter the object.
- Decide explicitly whether values such as the string '90' should be converted or rejected.

6.3 @final and typing.final

The typing.final decorator communicates that a class should not be subclassed or a method should not be overridden. Static type checkers such as mypy and Pyright can flag violations. Python itself generally does not enforce this rule at runtime.



```python
from typing import final

@final
class FixedConfiguration:
    def __init__(self, mode):
        self.mode = mode

class Student:
    @final
    def student_id_format(self):
        return "S followed by digits"

# A static type checker should flag subclassing FixedConfiguration
# or overriding student_id_format in a subclass.
```

Use @final as design documentation and static-analysis help, not as a security mechanism. In Python 3.11+, the decorator may set __final__ metadata on the object, but that metadata does not itself stop overriding.

6.4 Shape, Circle, and Rectangle Example



```python
from abc import ABC, abstractmethod
from math import pi

class Shape(ABC):
    @abstractmethod
    def area(self):
        pass

    @abstractmethod
    def perimeter(self):
        pass

class Circle(Shape):
    def __init__(self, radius):
        if radius <= 0:
            raise ValueError("Radius must be positive")
        self.radius = radius

    def area(self):
        return pi * self.radius ** 2

    def perimeter(self):
        return 2 * pi * self.radius
```

```python
class Rectangle(Shape):
    Tabnine | Edit | Test | Explain | Document
    def __init__(self, width, height):
        if width <= 0 or height <= 0:
            raise ValueError("Dimensions must be positive")
        self.width = width
        self.height = height

    Tabnine | Edit | Test | Explain | Document
    def area(self):
        return self.width * self.height

    Tabnine | Edit | Test | Explain | Document
    def perimeter(self):
        return 2 * (self.width + self.height)

shapes = [Circle(2), Rectangle(3, 4)]
for shape in shapes:
    print(type(shape).__name__, shape.area(), shape.perimeter())

```

This example demonstrates polymorphism: the same method names, area() and perimeter(), work for different shape objects. Each subclass supplies its own implementation, while Shape defines the common contract.

Important note: An abstract base class is useful when subclasses must implement a shared interface. Do not add abstraction only to make a small program look more complicated.

7. Operators — Part 1

7.1 Arithmetic Operators

Operator	Meaning	Example	Result
+	Addition	8 + 3	11
-	Subtraction	8 - 3	5
*	Multiplication	8 * 3	24
/	True division	8 / 3	2.666...
//	Floor division	8 // 3	2
%	Remainder	8 % 3	2
**	Exponentiation	2 ** 3	8

introduct_to_oop.py > ...

```python
total = 80 + 90 + 100
average = total / 3
print(average)  # 90.0

print(7 // 2)       # 3
print(-7 // 2)      # -4
print(7 % 2)        # 1
print(2 ** 3 ** 2)  # 512: exponentiation groups right-to-left
print(-2 ** 2)      # -4; interpreted as -(2 ** 2)

```

Division by zero raises ZeroDivisionError. The modulo operator follows Python's floor-division rules. Use parentheses when exponentiation and unary minus might be confusing.

7.2 Comparison Operators

Operator	Meaning	Example
==	Equal values	score == 100
!=	Not equal	score != 0
>	Greater than	age > 18
<	Less than	score < 50
>=	Greater than or equal	score >= 50
<=	Less than or equal	age <= 25

Comparison operators usually return True or False. Python supports chained comparisons, such as `0 <= score <= 100`, meaning both comparisons must be true.

```
score = 85
print(0 <= score <= 100)  # True
print(50 <= score < 90)   # True
print(score == "85")      # False
```

Important note: A single = assigns a value; == compares values.

7.3 Logical Operators

The logical operators are and, or, and not. Python's and and or short-circuit: the right-hand expression is evaluated only when needed. They return one of their operands, not necessarily a Boolean.

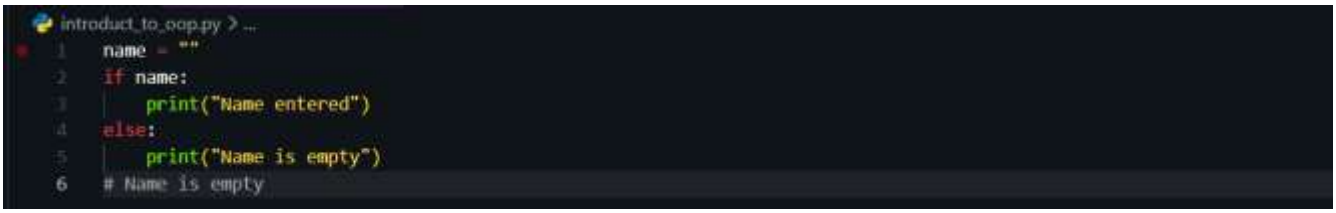
introduct_to_oop.py > ...

```python
age = 19
score = 82
eligible = age >= 18 and score >= 50
needs_help = score < 50 or age < 16
not_passed = not (score >= 50)
print(eligible, needs_help, not_passed) #True False False
```

For `A and B`, B is evaluated only if A is truthy. For `A or B`, B is evaluated only if A is falsy. This behavior can make a condition safe, but remember the result may be an operand.

Truthiness

Falsy values include False, None, numeric zero (0, 0.0, 0j), and empty strings or collections such as '', [], (), {}, and set(). Most other values are truthy.



7.4 Bitwise Operators

Bitwise operators operate on the bits of integers. They are used in flags, masks, and lower-level operations, rather than typical grade calculations.

Operator	Meaning	Example	Result
&	Bitwise AND	6 & 3	2 (110 AND 011 = 010)
|	Bitwise OR	6 | 3	7 (110 OR 011 = 111)
^	Bitwise XOR	6 ^ 3	5 (110 XOR 011 = 101)
~	Bitwise NOT	~6	-7; equals -x - 1
<<	Left shift	3 << 2	12
>>	Right shift	12 >> 2	3



Python integers have arbitrary precision, so describing ~x as flipping a fixed 8-bit or 32-bit pattern is misleading. For integers, `~x` equals `-x - 1`. Bitwise & differs from logical and.

8. Operators — Part 2

8.1 Assignment Operators

Operator	Meaning	Example
=	Assign	score = 80
+=	Add then assign	score += 5
-=	Subtract then assign	score -= 2
*=	Multiply then assign	score *= 2

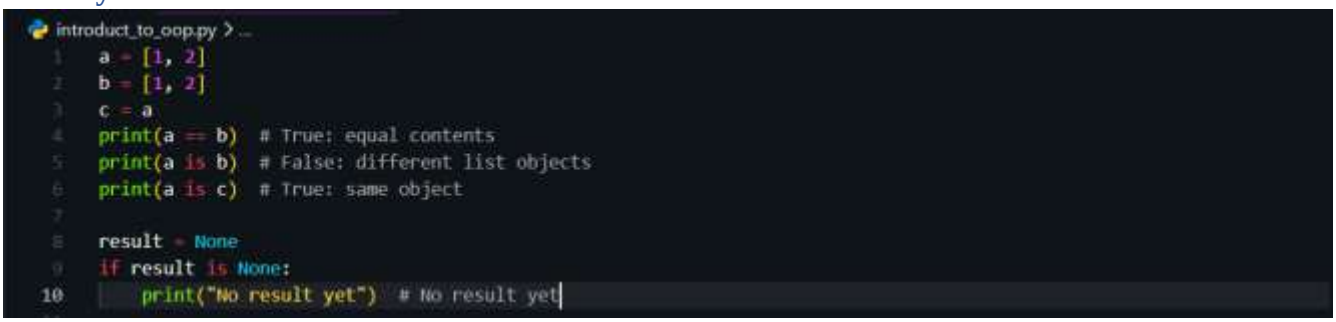
/=	Divide then assign	score /= 2
//=	Floor-divide then assign	count //= 2
%=	Remainder then assign	count %= 3
**=	Power then assign	value **= 2
&=, |=, ^=	Bitwise operation then assign	flags |= 0b0010
<<=, >>=	Shift then assign	bits <<= 1



Augmented assignment is concise, but for mutable objects it may update the existing object in place. For example, list += often extends the list in place, while list = list + [...] creates a new list.

8.2 Identity, Membership, and Conditional Expressions

Identity: is and is not



Use is None and is not None for None checks. Do not use is to compare numbers or strings: Python may reuse some immutable objects as an optimization, and that behavior is not a value-comparison rule.

Membership: in and not in



Membership checks list elements, dictionary keys, or string substrings depending on the object. Sets are generally efficient for membership checks.

Conditional expression

Python's conditional expression is `value_if_true if condition else value_if_false`. It is an expression that produces a value.

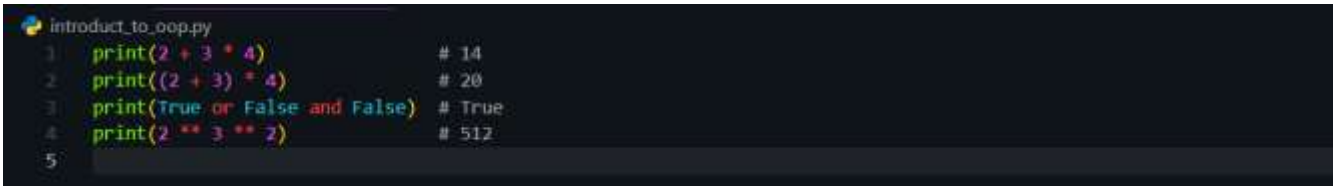
```
score = 72
status = "Pass" if score >= 50 else "Fail"
print(status)
```

Java uses `condition ? value1 : value2`; Python uses `value1 if condition else value2`. Python has no `? :` conditional syntax. Use a full if statement when the expression becomes hard to read.

8.3 Precedence and Associativity

Precedence controls which operators are evaluated first. Parentheses clarify intent and are usually preferable to memorizing every rule.

Simplified order: high to low	Operators / expressions
1	Parentheses, indexing, attribute access, function calls
2	Exponentiation ** (right-associative)
3	Unary +, -, ~
4	*, @, /, //, %
5	+ and -
6	<< and >>
7	&
8	^
9	|
10	Comparisons: <, <=, >, >=, !=, ==, in, not in, is, is not
11	not
12	and
13	or
14	Conditional expression: x if condition else y
15	lambda; assignment expression := has its own rules



Most arithmetic operators group left-to-right; exponentiation groups right-to-left. Comparisons can chain, so `1 < x < 10` is a single chained comparison.

I mportant note: For complicated expressions, add parentheses or split the expression into named variables. Readability is more valuable than saving a few characters.

9. Decision-Making

9.1 if, if-else, and if-elif-else

An if statement executes its indented body only when the condition is truthy. Python uses a colon and indentation to mark a block.

introduct_to_oop.py > ...

```
score = 85
if score >= 50:
    print("The student passed.")
else:
    print("The student failed.")

```

Use if-elif-else when multiple alternatives exist. Conditions are checked from top to bottom, and only the first matching branch runs.

introduct_to_oop.py > ...

```
score = 93

if score < 0 or score > 100:
    grade = "Invalid"
elif score >= 97:
    grade = "A+"
elif score >= 90:
    grade = "A"
elif score >= 80:
    grade = "B"
elif score >= 70:
    grade = "C"
elif score >= 60:
    grade = "D"
elif score >= 50:
    grade = "E"
else:
    grade = "F"

print(grade)  # A

```

Order matters. Check invalid scores first, then the highest grade thresholds. If `score >= 90` is checked before `score >= 97`, a score of 98 gets A and the A+ branch can never run.

Boundary conditions

Score	Grade in this example	Reason
-1	Invalid	Below 0
0	F	Valid, below 50
49.99	F	Below pass mark
50	E	Inclusive pass threshold
59.99	E	Below 60
60	D	Inclusive boundary
70	C	Inclusive boundary
80	B	Inclusive boundary

90	A	A+ starts at 97
96.99	A	Below A+ threshold
97	A+	Inclusive A+ threshold
100	A+	Maximum valid score
100.01	Invalid	Above 100

The grade scale is an example and may differ from your course. The essential skill is to define thresholds precisely and test the boundary values.

9.2 Compound Conditions and Validation

introduct_to_oop.py > ...

```python
age = 19
score = 85
attendance = 0.92

eligible = age >= 18 and score >= 50 and attendance >= 0.80
if eligible:
    print("Eligible")
else:
    print("Not eligible")

if (score >= 50 and attendance >= 0.80) or age >= 21:
    print("Meets one eligibility route")
```

Use and when every requirement must be true. Use or when at least one condition is enough. Parentheses can make compound conditions easier to understand.

9.3 Nested if Statements

A nested if is an if statement inside another if. It expresses dependent decisions, but deep nesting can become hard to read.

introduct_to_oop.py > ...

```python
logged_in = True
role = "teacher"
has_permission = True

if logged_in:
    if role == "teacher":
        if has_permission:
            print("Open grade entry")
        else:
            print("Permission denied")
    else:
        print("Teacher role required")
else:
    print("Please log in")
```

Early returns, helper functions, or carefully combined conditions can sometimes reduce nesting. Do not simplify conditions if the resulting logic becomes harder to understand.

9.4 Multiple-Branch Selection with match-case

Python 3.10 introduced structural pattern matching. For simple menu values, match-case and if-elif-else are both reasonable.

```python
choice = "2"

match choice:
    case "1":
        print("Add student")
    case "2":
        print("View students")
    case "3":
        print("Search students")
    case "0":
        print("Exit")
    case _:
        print("Invalid choice")
```

The `_` pattern is a catch-all case, similar to default in some languages. Cases are tested in order. In Python versions before 3.10, use if-elif-else.

```python
choice = input("Choose 1, 2, or 0: ").strip()

if choice == "1":
    print("Add student")
elif choice == "2":
    print("View students")
elif choice == "0":
    print("Exit")
else:
    print("Invalid choice")
```

input() returns a string. Compare it with strings such as '1', or convert it carefully and handle ValueError if the user may type non-numeric input.

10. Loops and Loop Control

10.1 The while Loop

A while loop repeats as long as its condition is truthy. It is useful when the number of repetitions is not known in advance.

```python
attempts = 0
while attempts < 3:
    print("Attempt", attempts + 1)
    attempts += 1
```

The condition is checked before each iteration, so a while loop can run zero times. Ensure that the condition eventually becomes false or that break can be reached; otherwise the loop may run forever.

Input validation with while

```python
while True:
    text = input("Enter a score from 0 to 100: ").strip()
    try:
        score = float(text)
    except ValueError:
        print("Please enter a number.")
        continue

    if 0 <= score <= 100:
        break
    print("Score must be between 0 and 100.")

print("Accepted score:", score)
```

"continue" restarts the loop after invalid text. break exits once a valid score is entered. A production program may also need to handle cancelled input or end-of-file.

10.2 The for Loop

A Python for loop iterates over items in an iterable: a list, tuple, string, dictionary, set, file, range, or another iterable object.

```python
students = ["Dara", "Malis", "Sokha"]
for name in students:
    print(name)

for number in range(1, 5):
    print(number)  # 1, 2, 3, 4
```

range(start, stop, step) generates integers; stop is excluded. `range(5)` gives 0–4, `range(1, 6)` gives 1–5, and `range(10, 0, -2)` gives 10, 8, 6, 4, 2.

Iterating through dictionaries

```python
student = {"name": "Dara", "age": 19, "score": 88}
for key in student:
    print(key, student[key])

for key, value in student.items():
    print(key, value)
```

```
Problems  Debug Console  Output  Ports  Terminal                                  Code

PS C:\a_University\I3 ITC\Plus Study For the DS\Python\OOP> python -u "c:\a_University\I3 ITC\Plus Study For the DS\Python\OOP\i
.py"
name Dara
age 19
score 88
name Dara
age 19
score 88
```

A dictionary loop iterates over keys by default. Use .items() for key-value pairs, .keys() for keys, or .values() for values.

Calculating an average

```python
scores = [80, 90, 75, 95]
if scores:
    average = sum(scores) / len(scores)
else:
    average = None
print(average)  #85.0
```

The empty-list check avoids division by zero. Built-ins sum() and len() are usually clearer than manually maintaining a total and count.

10.3 No Native do-while Loop

Python has no native do-while statement. A do-while loop in Java runs the body at least once and checks the condition afterward. Python can reproduce this with while True and break.



The body runs at least once because the condition is always True; break decides when to stop. This pattern avoids duplicating the first operation and the loop body.

10.4 break, continue, pass, and Loop else

Statement	Effect	Typical use
break	Exits the nearest enclosing loop immediately.	Stop searching after finding a match.
continue	Skips the rest of this iteration and starts the next.	Skip invalid or unwanted items.
pass	Does nothing; execution continues normally.	Placeholder where syntax requires a statement.



The example skips invalid scores, then stops at the first valid score of at least 90. It prints 95 and does not inspect later values.

Loop else

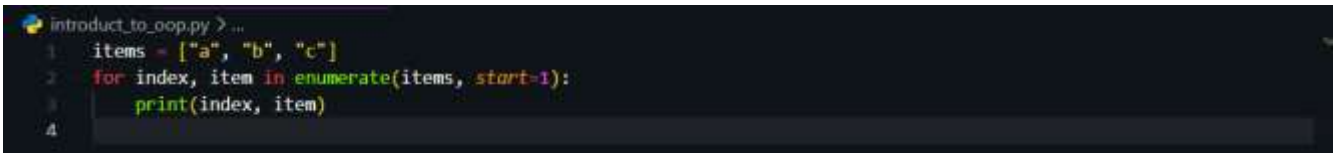
A for or while loop can have an else clause. The else block runs if the loop finishes normally, but not if break exits it.



The else belongs to the loop, not the if. If a match triggers break, the loop's else is skipped. If the loop ends without break, else runs.

Common loop mistakes

- Off-by-one errors: range(1, 5) stops at 4, not 5.
- Infinite while loops caused by a condition that never changes.
- Modifying a list during iteration can skip items or create confusing behavior.
- Using break to skip one item; use continue for that.
- Using pass as if it skips an iteration; pass does nothing.
- Assuming loops always run once; for and while can run zero times.
- Prefer `for item in items` when the index is unnecessary; use enumerate(items) when both index and item are needed.



11. Python and Java: Important Differences

Topic	Python	Java
Typing	Dynamically typed; names refer to objects.	Statically typed; declarations or inferred compile-time types.
Variable declaration	Usually assign directly: age = 19.	Often declare type: int age = 19;
Blocks	Indentation and colon define blocks.	Braces define blocks; semicolons commonly end statements.
Boolean literals	True and False.	true and false.
No-value marker	None.	null.
Strings	Immutable str; single or double quotes.	Immutable String; literals usually use double quotes.
Collections	Built-in list, tuple, dict, set.	Arrays and collection types such as ArrayList, HashMap, HashSet.
Tuple	Built-in immutable sequence.	No direct general-purpose core tuple equivalent; records/classes may serve similar needs.

Map/dictionary	dict.	Map interface and implementations such as HashMap.
Access control	Conventions, name mangling, and interfaces; not strict private fields.	public, protected, package-private, private.
Static method	@staticmethod; no automatic self or cls.	static method; no this argument.
Class method	@classmethod receives cls; supports subclass-aware construction.	No exact direct equivalent; static methods or constructors handle related tasks.
Initialization	__new__ creates object; __init__ initializes it.	Constructor uses class name.
Abstract classes	ABC and @abstractmethod.	abstract class and abstract methods.
Contracts	ABC and typing.Protocol are common.	Interfaces and abstract classes are common.
Final	typing.final is mainly a static type-checking hint.	final is a language modifier enforced by Java rules.
Conditional expression	value_if_true if condition else value_if_false.	condition ? value1 : value2.
Logical operators	and, or, not.	&&, ||, !.
Identity and equality	== compares values; is tests identity.	== compares primitive values or object identity; equals() commonly compares object values.
Collection loop	for item in items:	for (Type item : items)
Counter loop	for i in range(5):	for (int i = 0; i < 5; i++)
do-while	No native do-while; use while True and break.	Native do { ... } while (condition);
Exceptions	try/except; no checked exceptions.	try/catch; Java has checked exceptions.
Function arguments	Default values, keyword arguments, flexible argument forms.	Overloading is common; ordinary method calls do not use Python-style keyword arguments.

Conditional expression comparison

```python
# Python
score = 30
status = "Pass" if score >= 50 else "Fail"
```

Equivalent Java-style expression:

```java
score = 30
String status = (score >= 50) ? "Pass" : "Fail";
```

Loop comparison

```python
names = []
for name in names:
    print(name)
```

Equivalent Java enhanced for loop:

```java
for (String name : names) {
    System.out.println(name);
}
```

The goal is not to translate each line word-for-word. Learn the idioms of each language and write code that is clear and correct in that language.

13. Common Pitfalls and Debugging Checklist

13.1 Data Types and Literals

- Confusing 5 / 2 (2.5) with 5 // 2 (2).
- Writing true, false, or none instead of True, False, and None.
- Assuming {} is an empty set; it is an empty dictionary.
- Writing ('only item') and expecting a tuple; use ('only item',).
- Assuming b = a copies a list.
- Using a mutable class attribute for data that should be separate for each instance.
- Expecting every decimal float calculation to be exact.

13.2 Variables and OOP

- Using a local variable outside its function, causing NameError.
- Forgetting self in an instance method or when accessing instance attributes.
- Confusing class attributes with instance attributes after shadowing a name.
- Assuming _balance is enforced as private.
- Assuming __pin is secure; name mangling is not encryption.
- Using @staticmethod when the method needs self or cls.
- Naming a class-method first parameter self instead of the conventional cls.
- Instantiating an abstract class before implementing all abstract methods.
- Assigning to a property inside its own setter and causing recursion.
- Assuming @final prevents subclassing at runtime.

13.3 Operators and Conditions

- Using = instead of == in a comparison.
- Using is to compare numbers or strings instead of ==.
- Checking score >= 90 before score >= 97, making A+ unreachable.

- Failing to validate scores outside 0–100.
- Using and where or is required, or the reverse.
- Assuming and/or always return Boolean values.
- Confusing logical and bitwise operators.
- Relying on precedence instead of using parentheses for clarity.

13.4 Loops

- Forgetting that range excludes its stop value.
- Writing a while loop whose condition never changes.
- Using break instead of continue, or pass instead of continue.
- Assuming loop else runs even after break.
- Assuming for only works with lists; it works with any iterable.
- Modifying a collection during iteration and skipping elements.

13.5 Practical Debugging Checklist

1. Read the last line of a traceback to identify the exception type and message.
2. Check the exact line number shown in the traceback.
3. Inspect values and types using print(value) and type(value).
4. Test edge cases: empty collections, zero, negative values, and threshold boundaries.
5. Check indentation and colons in if, for, while, class, and function definitions.
6. Use descriptive names and split long expressions into smaller steps.
7. Reproduce the error with the smallest example possible.
8. After fixing it, rerun the original test and nearby boundary tests.

14. Practice Exercises

Try each task before looking back at the examples. They cover all major concepts in the report.

Exercise 1 — Types and literals

9. Create variables for a student's name, age, score, enrollment status, and missing email.
10. Create a list of subjects, a tuple of latitude and longitude, a student dictionary, and a set of unique course names.
11. Print each value and its type.
12. Explain why {} and set() are different.

Exercise 2 — Variables and attributes

13. Create a Student class with class attribute school_name and instance attributes name and age.
14. Create two students with different names and ages.
15. Change the class school name and observe both objects.
16. Assign a new school_name through only one instance and explain what happens.

Exercise 3 — Access conventions and methods

17. Create BankAccount with owner, _balance, and __account_code.
18. Add deposit() that rejects zero and negative deposits.
19. Add a @staticmethod that checks whether an amount is positive.
20. Add a @classmethod alternative constructor that accepts a dictionary.

Exercise 4 — Abstract class and property

21. Create an abstract Shape class with area().
22. Implement Circle and Rectangle.
23. Reject negative dimensions.
24. Create a score property that accepts only numbers between 0 and 100.

Exercise 5 — Operators

25. Calculate the average of three scores.
26. Use // and % to find quotient and remainder.
27. Show == versus is using two different lists with the same elements.
28. Check whether a name is in a list.
29. Rewrite a Java conditional expression in Python.

Exercise 6 — Decisions

30. Write a pass/fail program with 50 as the inclusive pass mark.
31. Write a letter-grade function and test 49, 50, 89, 90, 96, 97, and 100.
32. Reject scores below 0 and above 100.
33. Write a menu with match-case and rewrite it with if-elif-else.

Exercise 7 — Loops

34. Use for to print student names.
35. Calculate the average score, including the empty-list case.
36. Search for a student using break and loop else.
37. Use continue to skip invalid scores.
38. Use while True to ask until the user enters a valid score.

Exercise 8 — Reflection

Explain in your own words: (1) list versus tuple, (2) instance versus class attribute, (3) static method versus class method, (4) abstraction versus encapsulation, (5) == versus is, (6) break versus continue versus pass, and (7) one important Python–Java difference.

15. Conclusion and Quick Reference

Python's built-in types, object model, operators, conditions, and loops are the foundations of larger programs. Good code uses the simplest structure that expresses the logic clearly, validates important data, and tests boundary cases. OOP features such as properties and abstract base classes can make code easier to maintain when they solve a real design problem.

Need	Use / remember
Whole number	int
Decimal approximation	float
Truth value	True / False
Text	str
Mutable sequence	list
Immutable sequence	tuple
Key-value mapping	dict

Unique elements	set; empty set is set()
No value	None; check with is None
Instance attribute	self.attribute
Class attribute	defined in class body
Internal-use convention	_name
Name mangling	__name inside a class
Static method	@staticmethod; no automatic self or cls
Class method	@classmethod; first parameter cls
Abstract method	@abstractmethod in an ABC
Validated attribute interface	@property and @name.setter
Final hint	typing.final; mainly static checking
Conditional branches	if / elif / else
Pattern matching	match / case, Python 3.10+
Iterate over items	for item in iterable
Repeat while true	while condition
Exit nearest loop	break
Skip current iteration	continue
Placeholder statement	pass
Compare values	==
Check object identity	is
Membership	in / not in
Conditional expression	value1 if condition else value2